Complexity Analysis & Complete Search

Know how fast your idea is *before* you code it — then try everything

Learning goals

- Estimate running time with Big-O and turn constraints into an algorithm budget
- Analyze loops, nested loops, logarithmic loops, and amortized work
- Design **complete search** (brute force) solutions:
  - nested loops over candidates
  - all subsets (bitmasks)
  - all permutations (`next_permutation`)
- Cut the search space by reducing dimensions and pruning
- Use brute force as a **checker** for faster solutions

Motivation: AtCoder ABC085 C — Otoshidama

N bills (10000, 5000, 1000 yen) total Y yen. Find counts (x, y, z), or print `-1 -1 -1`.
N ≤ 2000.

- Try all x, y, z ≤ N: 2001^3 ≈ 8·10^9 — **too slow**
- But z = N − x − y is forced! Try all x, y: 2001^2 ≈ 4·10^6 — **fast**

Same idea, smarter enumeration. That's this lecture.

Big-O in one slide

f(n) = O(g(n)) if f(n) ≤ c·g(n) for large n.

- Drop constants and lower-order terms: $3n^2 + 10n + 7 = O(n^2)$
- Counts the **growth rate**, not exact time
- In CP: plug the max n into g and compare to ~10^8 per second

g(n)	n = 10^5
n	10^5
n log n	1.7·10^6
n^2	10^10 ×

Analyzing loops

```
for (int i = 0; i < n; i++)              // O(n)
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)      // n(n-1)/2 = O(n^2)
for (int i = 1; i < n; i *= 2)           // O(log n)
for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j += i)      // n/1 + n/2 + ... = O(n log n)
```

Last one: harmonic series H(n) ≈ ln n. Shows up in sieves (Week 10).

Amortized analysis

Total cost over all operations, divided by the number of operations.

```
int j = 0;
for (int i = 0; i < n; i++) {
    while (j < n && ok(i, j)) j++;      // looks like O(n^2)...
}
```

`j` only increases, at most n times **in total** → whole loop is O(n).
(Basis of two pointers, Week 6.) Same argument: `vector::push_back` is O(1) amortized.

From constraints to algorithm

n ≤	Budget	Typical technique
10	n! · n	all permutations
20	2^n · n	all subsets, bitmask
40	2^(n/2)	meet in the middle
500	n^3	triple loop, Floyd–Warshall
5000	n^2	double loop, O(n^2) DP
10^6	n log n	sort, binary search, segment tree
10^18	log n	math, binary search, fast power

Constant factors matter (a bit)

- 10^8 "simple" ops ≈ 1 s; but `map` ops cost ~10–50× an array access
- Cache-friendly loops (row-major traversal) can be several × faster
- Memory: 256 MB ≈ 6.4·10^7 `int`s or 3.2·10^7 `long long`s
- When close to the limit: count the real constant, not just the O

Complete search

> Enumerate every candidate solution, check each, keep the best.

- Always correct if the enumeration is complete
- The question is only: **is it fast enough?**
- Often the right answer when constraints are tiny
- Even when too slow, it's the reference to test a clever solution against

Three shapes: **loops**, **subsets**, **permutations**.

Shape 1: nested loops

ABC085 C — enumerate x and y, derive z:

```
for (int x = 0; x <= n; x++)
    for (int y = 0; x + y <= n; y++) {
        int z = n - x - y;
        if (10000 * x + 5000 * y + 1000 * z == Y) { /* found */ }
    }
```

Reduce dimensions: if one variable is determined by the others, don't loop over it.

Shape 2: all subsets via bitmasks

Each integer `mask` in [0, 2^n) encodes a subset: bit i set ⇔ item i chosen.

```
n = 3   mask  bits   subset
        0     000    {}
        5     101    {0, 2}
        7     111    {0, 1, 2}
```

```
for (int mask = 0; mask < (1 << n); mask++)
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) { /* item i is in the subset */ }
```

O(2^n · n). Good for n ≤ 20. Bit tricks in depth: Week 9.

Example: CSES 1623 — Apple Division

Split n ≤ 20 apples (weights ≤ 10^9) into two groups, minimize difference of sums.

- Try every subset as group 1; group 2 is the rest
- diff = |total − 2·sum(group 1)|
- 2^20 · 20 ≈ 2·10^7 ✓

Apple Division — code

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> p(n);
    long long total = 0;
    for (auto &x : p) { cin >> x; total += x; }
    long long best = LLONG_MAX;
    for (int mask = 0; mask < (1 << n); mask++) {
        long long s = 0;
        for (int i = 0; i < n; i++)
            if (mask >> i & 1) s += p[i];
        best = min(best, llabs(total - 2 * s));
    }
    cout << best << '\n';
}
```

Shape 3: all permutations

```
vector<int> p = {0, 1, 2, ..., n-1};        // must start SORTED
do {
    // process permutation p
} while (next_permutation(p.begin(), p.end()));
```

- n! permutations, each step O(n) worst / O(1) amortized → n ≤ 10 or 11
- Works with duplicates: generates each **distinct** arrangement once

Example: CSES 1622 — Creating Strings

Print all distinct permutations of a string ($|s| \le 8$), sorted.

```
string s; cin >> s;
sort(s.begin(), s.end());
vector<string> out;
do out.push_back(s); while (next_permutation(s.begin(), s.end()));
cout << out.size() << '\n';
for (auto &t : out) cout << t << '\n';
```

Sorted first ⇒ output is in lexicographic order, duplicates skipped.

Example: traveling through all orders

Given n ≤ 8 cities and a distance matrix, find the shortest route starting at 0 visiting all.

- Fix city 0 first, permute the rest: (n−1)! = 5040 orders, each O(n)
- Week 26 does n ≤ 20 with bitmask DP in O(2^n · n²)

Complete search is the baseline every clever algorithm improves on.

Choosing what to enumerate

CF 1097B Petr and a Combination Lock: n ≤ 15 rotations, each ± a_i. Can the total be ≡ 0 (mod 360)?

- Enumerate the **sign choice** of each rotation: 2^15 = 32768 masks ✓

ABC128 C Switches: N, M ≤ 10. Enumerate switch states (2^N), check each bulb.

Ask: *what is the smallest set of decisions that determines everything else?*

Pruning

Stop exploring a partial candidate as soon as it cannot lead to a valid/better answer.

- Partial sum already exceeds the target → stop
- Partial cost ≥ best found so far → stop (branch and bound)
- Symmetry: fix the first element, or require chosen indices to increase

Requires **recursive** generation (Week 8: backtracking).

Brute force as a test oracle

```
gen.cpp   -> random small input
brute.cpp -> slow, obviously correct
fast.cpp  -> your real solution
loop: gen > in; brute < in > a; fast < in > b; diff a b || break
```

- Finds WA before the judge does
- We'll build a full stress-testing setup in Week 35

Complexity pitfalls

- `1 << n` with n ≥ 31 overflows `int` — use `1LL << n`
- Hidden O(n) inside a loop: `s.substr`, `vector` copy, `find` on vector, `erase` from front
- Multi-test: per-test O(maxN) work × t tests
- Recursion depth 10^6 → stack overflow (RE)
- "It's only 10^9 simple operations" — still ~1–3 s. Usually too slow

Practice problems

- CSES 1623 Apple Division · 1622 Creating Strings · 1624 Chessboard and Queens (preview)
- AtCoder ABC085 C Otoshidama · ABC128 C Switches · ABC051 B Sum of Three Integers · ABC045 C Many Formulas · ABC079 C Train Ticket
- Codeforces 1097B Petr and a Combination Lock · 550B Preparing Olympiad · 124B Permutations
- USACO 2019 Dec Bronze — Cow Gymnastics · USACO 2018 Jan Bronze — Lifeguards

Summary

- Constraints → budget → technique. Do this **before** coding
- Loops: multiply nested bounds; watch for logs and harmonic sums; use amortized arguments
- Complete search = enumerate loops / subsets / permutations, check each
- Make enumeration smaller: derive variables, enumerate decisions, prune
- Next week: **sorting & greedy** — when you don't need to try everything